

COMS3011A LAB 1

Curated Transcript of AI Usage

Planning, code generation, debugging and verification

Project: Full-stack Todo Application

Next.js, TypeScript, Prisma, SQLite and Vitest

Compiled from six exported ChatGPT branches

2 August 2026

This is a deliberately selective academic transcript. Repetitive navigation questions, basic Git-operation queries, short acknowledgements, empty export pages and duplicated branch prefixes were excluded. Minor spelling and punctuation were normalised without changing the meaning of the retained requests.

Contents and selection method

- Part I - Planning and design decisions
- Part II - Code generation and implementation
- Part III - Debugging and verification
- Appendix - Source map

Selection criteria

- Retained only exchanges that materially shaped the architecture, final code, tests or submission evidence.
- Removed low-value operational questions such as how to view commit history or reopen the folder.
- Condensed long answers to the relevant excerpts while preserving generated code and commands.
- Kept source filenames and page ranges so every excerpt remains traceable to the original exports.

Part I - Planning and design decisions

1. Interpreting the assignment and defining the deliverable

USER REQUEST - SPELLING NORMALISED

Review the lab brief and explain the task, the appropriate language and stack, and a practical way to approach the implementation.

AI RESPONSE - SELECTED EXCERPT

The assignment was interpreted as a full-stack local todo application rather than a static webpage. The required behaviours were identified as persistent task creation, editing, archiving, active and archived views, sorting by topic/status/due date, exactly three statuses, and a derived overdue warning. The response also highlighted that data must survive application restarts.

Source: chat 2.pdf, pp. 1-4.

2. Choosing the architecture and database model

AI RESPONSE - SELECTED EXCERPT

The recommended architecture used TypeScript and React/TSX in Next.js, Prisma as the database-access layer, SQLite for local persistence, CSS for presentation and Vitest for tests. A single Task table was proposed with an archivedAt timestamp rather than a deletion workflow. Overdue was explicitly treated as a calculated display state, not a stored column or fourth status.

```
Task
- id
- title
- description
- dueDate
- topic
- status: TODO | IN_PROGRESS | COMPLETE
- archivedAt: DateTime | null
- createdAt
- updatedAt
```

```
const overdue =
  task.dueDate < new Date() &&
  task.status !== "COMPLETE";
```

Source: chat 2.pdf, pp. 2-4.

3. Establishing the implementation sequence and risk controls

AI RESPONSE - SELECTED EXCERPT

The work was broken into coherent stages: initialise the project, build the schema and migration, implement persistent creation/listing, add editing and archiving, add sorting and overdue display, add deterministic tests, then complete documentation and clean-clone verification. The response prioritised database persistence and marker-visible behaviour before styling.

1. Project and repository setup
2. SQLite schema and migration
3. Persistent task creation and listing
4. Editing and status updates
5. Archiving and archived view
6. Sorting and overdue indication
7. Deterministic tests
8. Documentation and clean-clone validation

Source: chat 2.pdf, pp. 5-11.

4. Refining the interface without changing the functional design

USER REQUEST - SPELLING NORMALISED

Keep the title and description on the left, with due date and topic on the right, using a two-by-two input layout. Increase the spacing around the active-task list and sorting controls slightly.

AI RESPONSE - SELECTED EXCERPT

The response adjusted the form layout and spacing while preserving the server actions, database behaviour and sorting form. This kept presentation changes isolated from the application logic.

Compilation note: This is retained as an example of the student directing the design rather than accepting the first layout unchanged.

Source: chat 5.pdf, pp. 54-59.

Part II - Code generation and implementation

5. Generating the Prisma schema and migration workflow

AI RESPONSE - SELECTED EXCERPT

The generated schema restricted task status through an enum, kept archivedAt nullable and avoided any overdue field. The migration workflow formatted and validated the schema, created the SQLite migration and generated Prisma Client.

```
enum TaskStatus {
  TODO
  IN_PROGRESS
  COMPLETE
}

model Task {
  id          Int          @id @default(autoincrement())
  title       String
  description  String
  dueDate     DateTime
  topic       String
  status      TaskStatus @default(TODO)
  archivedAt  DateTime?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}
```

```
npx prisma format
npx prisma validate
npx prisma migrate dev --name create_task_table
npx prisma generate
```

Source: chat 2.pdf, pp. 28-31; caht 1.pdf, pp. 29-31.

6. Connecting Prisma and implementing persistent creation/listing

AI RESPONSE - SELECTED EXCERPT

A reusable Prisma client was generated with the better-sqlite3 adapter and a development global cache. A server action then validated the four required fields, parsed the due date, inserted the task and revalidated the page. The page queried active tasks from SQLite rather than storing them only in React state.

```
const adapter = new PrismaBetterSqlite3({
  url: process.env.DATABASE_URL ?? "file:./dev.db",
});

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({ adapter });

export async function createTask(formData: FormData) {
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
  const topic = formData.get("topic")?.toString().trim();
  const dueDateValue = formData.get("dueDate")?.toString();

  if (!title || !description || !topic || !dueDateValue) {
    throw new Error("All task fields are required.");
  }

  await prisma.task.create({
    data: { title, description, topic, dueDate: new Date(dueDateValue) },
  });

  revalidatePath("/");
}
```

Source: chat 3.pdf, pp. 30-34.

7. Adding editing, validated status changes and non-destructive archiving

AI RESPONSE - SELECTED EXCERPT

The generated update action validated the task ID, required fields, due date and TaskStatus enum before updating the database. Archiving preserved the row and set archivedAt. The page queried active and archived records separately and exposed only Todo, In-Progress and Complete in the edit form.

```
if (!Object.values(TaskStatus).includes(statusValue as TaskStatus)) {
  throw new Error("Invalid task status.");
}

await prisma.task.update({
  where: { id: taskId },
```

```

data: {
  title,
  description,
  topic,
  dueDate,
  status: statusValue as TaskStatus,
},
});

```

```

await prisma.task.update({
  where: { id: taskId },
  data: { archivedAt: new Date() },
});

```

Source: caht 4.pdf, pp. 38-45.

8. Implementing active-task sorting

AI RESPONSE - SELECTED EXCERPT

Sorting was implemented through a validated URL search parameter and a Prisma orderBy expression. Only the active list used the selected mode; archived tasks remained ordered by the time they were archived. Secondary due-date ordering kept results predictable when topics or statuses matched.

```

type SortOption = "dueDate" | "topic" | "status";

function getTaskOrder(sort: SortOption) {
  switch (sort) {
    case "topic":
      return [{ topic: "asc" }, { dueDate: "asc" }];
    case "status":
      return [{ status: "asc" }, { dueDate: "asc" }];
    default:
      return [{ dueDate: "asc" }];
  }
}

```

Source: caht 4.pdf, pp. 46-55.

9. Deriving and displaying overdue state

AI RESPONSE - SELECTED EXCERPT

The overdue feature was generated without changing the schema. A task is overdue when its due date is earlier than the current time and it is not complete. The result controls a visual card class and badge, while existing edit and archive forms remain intact.

```

export function isTaskOverdue(
  dueDate: Date,
  status: TaskStatus,
  now: Date = new Date(),
): boolean {
  return (
    dueDate.getTime() < now.getTime() &&
    status !== TaskStatus.COMPLETE
  );
}

```

```

<li className={overdue ? "task-card task-card-overdue" : "task-card"}>
  <h3>{task.title}</h3>
  {overdue && <p className="overdue-badge">Overdue</p>}
  {/* edit and archive forms retained */}
</li>

```

Source: chat 6.pdf, pp. 49-54; chat 5.pdf, pp. 60-62.

10. Extracting business logic for deterministic tests

AI RESPONSE - SELECTED EXCERPT

The overdue rule was moved into a dedicated library function with an optional now parameter. This allowed tests to use a fixed reference time rather than the real clock. Four unit tests covered past-due Todo, past-due In-Progress, completed tasks and future tasks.

```

const now = new Date("2026-07-30T12:00:00.000Z");
const past = new Date("2026-07-29T12:00:00.000Z");
const future = new Date("2026-07-31T12:00:00.000Z");

expect(isTaskOverdue(past, TaskStatus.TODO, now)).toBe(true);
expect(isTaskOverdue(past, TaskStatus.IN_PROGRESS, now)).toBe(true);
expect(isTaskOverdue(past, TaskStatus.COMPLETE, now)).toBe(false);
expect(isTaskOverdue(future, TaskStatus.TODO, now)).toBe(false);

```

Source: chat 5.pdf, pp. 58-63.

Part III - Debugging and verification

11. Correcting environment-file handling for a clean clone

USER REQUEST - SPELLING NORMALISED

Before committing, verify that the real .env and local SQLite database remain private, while a clean clone still receives the required database configuration.

AI RESPONSE - SELECTED EXCERPT

The response identified that the broad .env* ignore rule also excluded .env.example. The fix explicitly unignored .env.example, kept .env and dev.db out of Git, and used the example file to document DATABASE_URL for a clean clone.

```
# Environment files
.env*
!.env.example
```

```
DATABASE_URL="file:./dev.db"
```

```
git check-ignore -v .env.example
git status
```

Source: chat 2.pdf, pp. 29-33; chat 3.pdf, pp. 29-30.

12. Verifying persistence rather than temporary interface state

AI RESPONSE - SELECTED EXCERPT

The generated verification sequence created a task, refreshed the browser, stopped the development server, restarted it and confirmed that the task remained. The same persistence check was later applied to edited values. This demonstrated that the final behaviour depended on SQLite rather than transient component state.

```
npm run dev
# Create or edit a task
# Refresh the browser
# Stop with Ctrl+C
npm run dev
# Confirm the saved values remain
```

Source: chat 3.pdf, pp. 32-34; chat 4.pdf, pp. 41-45.

13. Validating the test strategy and disposable database requirement

AI RESPONSE - SELECTED EXCERPT

The final test review confirmed four overdue-rule tests and one database-behaviour test. The response specifically required the database test to use a temporary/disposable SQLite database rather than dev.db and to clean it up afterwards.

```
RUN v4.1.10
✓ tests/task-rules.test.ts (4 tests)
✓ tests/task-database.test.ts (1 test)
```

```
Test Files 2 passed (2)
Tests      5 passed (5)
```

Source: chat 6.pdf, pp. 63-66.

14. Interpreting final lint, type-check, build and test output

AI RESPONSE - SELECTED EXCERPT

The final verification was interpreted as successful: ESLint passed, TypeScript passed, the production build compiled and all five tests passed. The dynamic route marker was expected because the home page reads current task data from SQLite. Before committing, the response again checked that .env, dev.db and temporary test databases were not staged.

```
npm run lint
npm run build
npm test

git status
git add .
git commit -m "Add deterministic task rules and database tests"
git push
```

Source: chat 6.pdf, pp. 63-66.

Appendix - Source map

Source	Uploaded filename	Retained material
Chat 1	caht 1.pdf	Prisma schema explanation and database-model context.
Chat 2	chat 2.pdf	Assignment planning, architecture, migration and environment-file handling.
Chat 3	chat 3.pdf	Prisma connection, creation/listing and persistence verification.
Chat 4	caht 4.pdf	Editing, status validation, archiving and sorting.
Chat 5	chat 5.pdf	Interface refinement, Vitest setup and deterministic overdue tests.
Chat 6	chat 6.pdf	Overdue display and final lint/build/test verification.

Compilation decisions

- Excluded basic navigation, commit-history and folder-reopening questions.
- Excluded repeated branch ancestry and duplicated opening material.
- Normalised obvious spelling and punctuation errors in retained user requests.
- Preserved the project decisions, generated code, meaningful debugging and final verification evidence.